

Homework 2

Let's remind ourselves of the asymptotic order notation¹. Let $f(x)$ and $g(x)$ be functions of a positive x .

$$f(x) = \mathcal{O}(g(x))$$

when there is positive constant c such that,

$$f(x) \leq cg(x)$$

for all $x \geq n$. Function f is then stated as *big oh* of g . Similarly, f is *big omega* of g , i. e.,

$$f(x) = \Omega(g(x))$$

when there exist positive constants c, n such that for all $x \geq n$ we have,

$$f(x) \geq cg(x)$$

Lastly, if f is both big- $\mathcal{O}$ and big- Ω of g ,

$$\Omega(g(x)) = f(x) = \mathcal{O}(g(x))$$

Then such a tight bound is stated as f is big- Θ of g ,

$$f(x) = \Theta(g(x))$$

Question 1. Consider the following Java subroutine,

```

1 public static void conditionalWork(int n) {
2     for (int i = 0; i < n; i++)
3         if (Math.random() < 0.5)
4             taskA()
5         else
6             taskB()
7 }
```

- 1) If on a certain machine the function `taskA()` takes 2 seconds on each call in the loop and the function `taskB()` takes 4 seconds then for $n = 10$, what is the total number of *expected* seconds taken by the subroutine `public static void conditionalWork(int n)`?
- 2) The subroutine `conditionalWork(int n)` is ran on a much faster machine reducing the time taken by `taskA()` on each call down to $\frac{1}{25}$ th of a second and `taskB()` now takes $\frac{1}{5}$ th of a second. For $n = 10$, what is the new total number of seconds *expected* by the subroutine `conditionalWork(int n)`?
- 3) Assume that for any arbitrary n , `taskA()` takes $\log(n)$ steps while `taskB()` takes $2n^2$ steps. Let $T(n)$ be the total number of steps *expected* by `conditionalWork(int n)`, what is $T(n)$?
- 4) What is the *best expected asymptotic* runtime complexity written as $T(n) = \Omega(g(n))$?
- 5) What is the *worst expected asymptotic* runtime complexity written as $T(n) = \mathcal{O}(g(n))$?
- 6) If `taskB()` took $4\log(n)$ steps, what is the *tight expected asymptotic* runtime complexity written as $T(n) = \Theta(g(n))$?

Question 2. In the code snippet bellow, we see two ways of getting the tenth place digit of a Java `int n`. Give the runtime complexity of each method using the *big oh* notation. Justify your answer.

```

1 System.out.println(n % 10);
2 System.out.println(String.valueOf(n).charAt(String.valueOf(n).length() - 1));
```

¹Sometimes also referred to as the **Bachmann-Landau notation**.

Question 3. Consider the following functions,

$$a(x) = 2(x^3 + 1)(x^2 - 1) + 2$$

$$b(x) = 2x^2$$

$$\alpha(x) = x^2$$

$$\beta(x) = \pi x^2$$

- 1) Observe that $b(x) = \mathcal{O}(a(x))$ because $b(x) \leq a(x)$ past a certain $x = n$. What is the value of n in this case?
- 2) Observe that $\beta(x) = \mathcal{O}(\alpha(x))$ because there exists a c such that $\beta(x) \leq c\alpha(x)$ for all $x > 0$. What is the value of c in this case?

Question 4. Given in listing 1 is a Java subroutine that sorts an array of non-negative Java integers in ascending order.

```

1  public static void sortIntegers(int[] toSort) {
2      int i = 0, j = 0, k = 0, max = Integer.MIN_VALUE;
3      for (i = 0; i < toSort.length; i++)
4          max = toSort[i] > max ? toSort[i] : max;
5      int[] counts = new int[max + 1];
6      for (i = 0; i < toSort.length; i++)
7          counts[toSort[i]]++;
8      for (i = 0; i < counts.length; i++)
9          for (j = 0; j < counts[i]; j++)
10         toSort[k++] = i;
11 }

```

LISTING 1—Linear time algorithm for sorting unique integers.

Assume that the length of the input parameter `int[] toSort` is $n + 1$, the $\max(\text{toSort}) \leq n$ and that `toSort` has unique elements. Let $T(n)$ be the worst and the average case complexity of the algorithm's runtime and $S(\text{toSort})$ be the worst case complexity of the memory-space.

- 1) What is $T(n)$?
- 2) What are $\mathcal{O}(T(n))$, $\mathcal{O}(S(\text{toSort}))$?
- 3) Look up and state the *big- $\mathcal{O}$* of the average case complexity of Java's built-in `Arrays.sort(int[])`. Is this better than $\mathcal{O}(T(n))$ from the previous step of this question?

Question 5. Given bellow is the Java implementation of the sieve of Eratosthenes. This particular implementation marks all the composite numbers as `true` since Java allocates `false` to all the elements of a newly created boolean arrays.

```

1  public static void eratosthenes(boolean[] toSieve) {
2      toSieve[0] = true;
3      toSieve[1] = true;
4      for (int i = 2; i < Math.sqrt(toSieve.length); i++)
5          if (!toSieve[i])
6              for (int j = i*i; j < toSieve.length; j += i)
7                  toSieve[j] = true;
8  }

```

Give an upper-bound on the runtime complexity of this particular implementation of the sieve of Eratosthenes. The better upper-bound you give, the more credit you get. Justify your answer.

SUBMISSION INSTRUCTIONS

Submit a PDF file with your answers.